

Atividade Prática Assíncrona

Geomerez Raduan de Oliveira Bandeira

Repositório: https://github.com/raduanoliveira/desenvolvimento_com_ia_aula_assincrona

Este relatório apresenta os resultados das Etapas 1 a 4 da prática assíncrona de harness e arquitetura, realizadas no laboratório ia-dev-lab. A Etapa 1 compara dois modos de autonomia do agente e descreve um hook de proteção. A Etapa 2 compara implementação com TDD e sem TDD e registra o uso do plugin Superpowers em uma feature nova. A Etapa 3 guarda o transcript da sessão e define um checkpoint humano obrigatório. A Etapa 4 resume a arquitetura a partir do código e registra a decisão de manter o ToDo no tamanho atual. Tudo isso sob o AGENTS.md e com testes no Docker.

1 INTRODUÇÃO AO PROJETO E À TAREFA. Mantive o monorepo com API Laravel, interface em React com TypeScript, Context API e Material UI, e execução exclusiva no Docker. O AGENTS.md determina Service por caso de uso, controller restrito ao HTTP, persistência por repositório, tipos de teste separados e vedação a pular camada. Essas regras valem para o agente em qualquer modo de autonomia. A tarefa da Etapa 1, repetida nos dois modos com o mesmo prompt e o mesmo modelo (Composer), foi acrescentar prazo opcional (due_date, apenas data) e um toaster, ao entrar no painel autenticado, para tarefas ativas, não arquivadas e não concluídas cujo prazo é o dia seguinte. O recorte é pequeno de propósito: o que se compara não é o tamanho da feature, e sim o comportamento do harness.

2 COMPARAÇÃO ENTRE OS MODOS AUTO E PLAN. No modo Auto (feature/prazo-auto), o agente avançou com aceite contínuo de edições e concluiu em cerca de trinta minutos, de ponta a ponta, com retrabalho posterior em testes: o risco percebido foi médio. No modo Plan (feature/prazo-plan), revisei o plano escrito e só então autorizei a implementação, em cerca de quarenta minutos após o aceite; o escopo já estava fechado e o risco percebido foi baixo. Em testes, o Auto encerrou com 54 no backend e 25 no frontend; o Plan, com 54 no backend e 26 no frontend. Prefiro o Auto só no tempo, porque concluiu a mesma tarefa sem etapa prévia de planejamento. Prefiro o Plan no risco e nos testes: a cobertura é equivalente, com um caso a mais no frontend, e não precisei corrigir falhas depois. As duas execuções produziram o mesmo comportamento funcional e respeitaram o AGENTS.md. Segui com a branch Plan, por maior controle. A branch Auto permanece no repositório como evidência.

3 HOOK DE PROTEÇÃO. Configurei o hook em .cursor/hooks.json e .cursor/hooks/block-db-destruction.mjs, nos eventos beforeShellExecution e preToolUse. O risco escolhido foi destruir o banco do laboratório: migrate:fresh, migrate:refresh, migrate:reset, db:wipe, schema:drop ou remoção de database.sqlite. Comandos seguros, como migrate e a bateria de testes, permanecem liberados. A validação teve oito casos locais aprovados e uma tentativa, no agente, de rodar comando com migrate:fresh. O Cursor recusou a ação e exibiu a mensagem do harness.

4 IMPLEMENTAÇÃO COM TDD E SEM TDD. Na branch `feature/tdd-guardrail`, a tarefa A filtrou a lista por `all`, `pending` ou `done`. A ordem foi teste, falha observada e código mínimo: os testes vieram antes (Service, API com 422 para status inválido, gateway e componente). O commit 49ca1d1 é o Red; o 48a55fe, o Green. A regra ficou no Service. Passaram 58 testes no backend e 28 no frontend. A aderência ao `AGENTS.md` foi alta, com TDD explícito, e o risco residual ficou baixo no escopo do filtro. A tarefa B, sem teste prévio, editou o título no caminho feliz: as camadas existiam, mas não houve Red nem testes novos da feature. A suíte antiga continuou verde sem cobrir o rename, o dono da tarefa nem o título vazio na rota. A aderência ao `AGENTS.md` foi parcial — camadas corretas e TDD ignorado — e o risco residual ficou médio, porque uma regressão do rename pode passar.

5 SUPERPOWERS. Usei o plugin Superpowers de ponta a ponta na prioridade da tarefa (`high`, `medium` ou `low`): criar, editar e mostrar na lista. O recorte foi uma mudança delimitada no `ToDo` já existente. O agente apresentou o desenho — coluna no banco, Service de criação, Service novo de atualização, `PATCH /api/tasks/{id}/priority` e seletor na tela — e só implementou depois da minha aprovação. Em seguida escreveu o teste que falhou e o código mínimo até a suíte voltar a verde. A evidência está na branch `feature/task-priority`, commit 0b75d75. No Docker, passaram 67 testes no backend e 32 no frontend. O Superpowers não substituiu o `AGENTS.md`: reforçou a ordem, com checkpoint humano no desenho, Red antes do Green e verificação antes de declarar pronto.

6 OBSERVABILIDADE E CHECKPOINT HUMANO. Exportei o log da sessão do agente, ao longo desta prática, para `ia-dev-lab/docs/sessao-log.md`. O arquivo reúne as falas das sessões do Cursor sem reescrever o diálogo; as chamadas de ferramenta ficaram de fora só para o texto caber no repositório. O checkpoint humano obrigatório deste `ToDo` é parar *antes de criar ou aplicar uma migration*: arquivo novo em `database/migrations/` ou `php artisan migrate`, inclusive via Docker. Isso não se confunde com o hook da Etapa 1: o hook recusa destruir o banco (`migrate:fresh`, `db:wipe` e afins); o checkpoint decide se o schema pode mudar. A regra entrou no `AGENTS.md` e na rule `checkpoint-migration.mdc`. A simulação ocorreu em 6 de setembro de 2026, no modo Plan da branch `feature/prazo-plan`: o agente apresentou o plano da feature de prazo com migration nova e não aplicou o schema enquanto eu não aceitei. A migration em jogo foi `2026_09_06_120000_add_due_date_to_tasks_table` (coluna opcional `due_date`). A decisão foi **aprovar**: autorizei o plano, não editei o recorte da coluna e não rejeitei. Só então a implementação e o `migrate` seguiram. Assumi o papel de responsável pelo schema e pelos dados do laboratório, não só de revisor de código: o SQLite do Docker é o estado real das tarefas, e o hook não pergunta se a coluna nova é desejável. Houve uma segunda parada do mesmo tipo na prioridade (`2026_09_06_141500_add_priority_to_tasks_table`), de novo com aprovação do desenho antes do código. O registro completo está em `ia-dev-lab/docs/harness-checkpoint.md`.

7 REVISÃO ARQUITETURAL. Pedi ao agente um resumo a partir do código, não de memória. O laboratório tem dois processos no Docker: API Laravel na porta 8000 e tela React na 5173. No backend, os módulos são tarefa e identidade. A apresentação (controllers, Form Requests, Resources) chama um Service por caso de uso; a persistência fica atrás de `TaskRepositoryInterface` e `UserRepositoryInterface`; o login usa `IdentityProviderInterface` com adaptador Socialite. No frontend, a tela fala com o Context

e o Context fala com taskApi ou authApi. O contrato entre os lados é REST com cookie de sessão, sem OpenAPI. A violação encontrada não era pular camada: era o contrato do repositório devolver o Model Eloquent Task, de modo que os Services importavam persistência no tipo e lançavam ModelNotFoundException. Decidi que o projeto **está no tamanho certo** quanto a processos — extrair um serviço de tarefas ou de login viraria acoplamento de rede, cookie e deploy, sem segundo consumidor — e **fechei o contrato da aplicação**: entidade Task em app/Domain, TaskNotFoundException no Service, Eloquent só no Adapter. Não houve migration. No Docker, passaram 68 testes no backend e 32 no frontend. O resumo completo está em `ia-dev-lab/docs/harness-arquitetura.md`.

8 CONSIDERAÇÕES FINAIS. Na Etapa 1, o modo Plan tornou-se a base do harness e o hook passou a recusar a destruição do banco. Na Etapa 2, o TDD deixou trilha auditável no filtro, enquanto a edição sem teste mostrou falsa segurança da suíte prévia. O Superpowers, na prioridade, repetiu o ciclo da tarefa A, com aprovação humana antes do código. Na Etapa 3, o transcript ficou no repositório e o checkpoint passou a exigir aceite humano antes de migration: o hook impede o wipe; o checkpoint decide se o modelo de dados muda. Na Etapa 4, a revisão a partir do código manteve o ToDo no tamanho certo (sem extrair processo) e fechou o contrato da tarefa com entidade de domínio, sem migration. As três features permanecem no mesmo laboratório, com as mesmas camadas e a mesma bateria executada no Docker. O controle do harness, nestas etapas, não esteve em gerar mais código, e sim em decidir o modo de autonomia, recusar ação destrutiva, exigir teste vermelho quando a regra pedia TDD, recusar implementação antes do desenho aprovado, parar antes de alterar o schema e recusar fatiar a arquitetura sem critério de acoplamento.